



รายงาน

เรื่อง Robot Car Contest

จัดทำโดย

1. นาย พิทวัส เฉิน 67011509
2. นาย ชاکร บุญชงาม 67010199

เสนอ

อาจารย์ จิระศักดิ์ สิทธิกร

รายงานฉบับนี้เป็นส่วนหนึ่งของรายวิชา Introduction To Computer Engineering

ภาคเรียนที่ 1 ปีการศึกษา 2567

สถาบันเทคโนโลยีพระจอมเกล้าเจ้าคุณทหารลาดกระบัง

สารบัญ

หัวเรื่อง

สารบัญ	ก
สารบัญภาพ	ข
Concept การทำงานของ Robot car	1
Hardware design	2
Circuit diagram	4
หลักการทำงานของ Robot car Software design.....	5
IR Sensor	5
Robot car decision	6
PID Control.....	6
กรณีหยุดรถ.....	9
LDR sensor.....	9
Ultrasonic sensor.....	10
การสั่งการมอเตอร์.....	11
ปัญหาที่พบเจอ	12
Source code	13

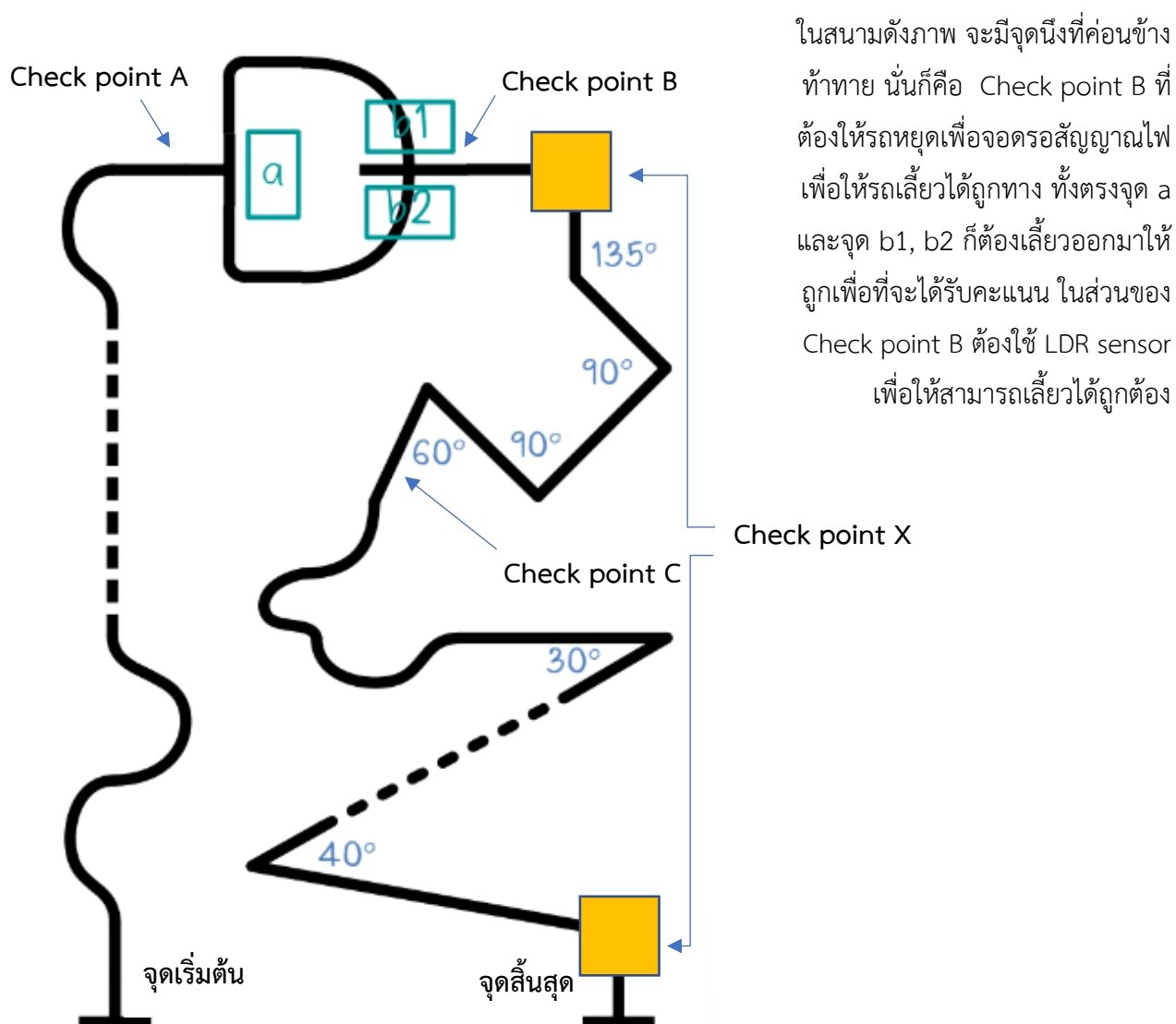
สารบัญภาพ

ภาพ

ภาพที่ 1 สนามแข่งขัน	1
ภาพที่ 2 Robot car	3
ภาพที่ 3 Circuit diagram.....	4
ภาพที่ 4 หลักการทำงานของ Robot car	5
ภาพที่ 5 หลักการทำงานของ IR sensor	6
ภาพที่ 6 หลักการของ PID คร่าวๆ.....	7
ภาพที่ 7 การจัดวาง IR sensor.....	8
ภาพที่ 8 ค่า Error ของแต่ละกรณี	8
ภาพที่ 9 สูตรคำนวณ PID และควบคุมมอเตอร์	8
ภาพที่ 10 แผนภาพ flowchart การทำงานของ LDR.....	9
ภาพที่ 11 แผนภาพ flowchart การทำงานของ Ultrasonic	10
ภาพที่ 12 ตัวอย่างการสั่งให้หยุดรถ	11
ภาพที่ 13 ตัวอย่างการสั่งให้รถไปข้างหน้า	11
ภาพที่ 14 Source code.....	13
ภาพที่ 15 Source code.....	14
ภาพที่ 16 Source code.....	15
ภาพที่ 17 Source code.....	16

Concept การทำงานของ Robot car

เริ่มจากหุ่นยนต์ตัวนี้จะทำการอ่านค่าระยะทางจาก Ultrasonic Sensor ก่อนหลังจากนั้นจะดูว่าค่าระยะทางถ้าน้อยกว่าหรือเท่ากับ 20 เซนติเมตร จะลดความเร็วรถลงและเมื่อใกล้สิ่งกีดขวางเข้าไปอีกจนมีค่าน้อยกว่า 5 เซนติเมตรจะหยุดรถ หลังจากนั้นให้มาอ่านค่าจาก IR Sensor แล้วนำค่าไปคำนวณ error เพื่อนำไปคำนวณค่า PID และนำค่าจากการคำนวณ PID มาปรับ ลด/เพิ่ม ความเร็วของมอเตอร์เพื่อให้รถสามารถเดินตามเส้นได้ ผ่านแต่ละ Check point

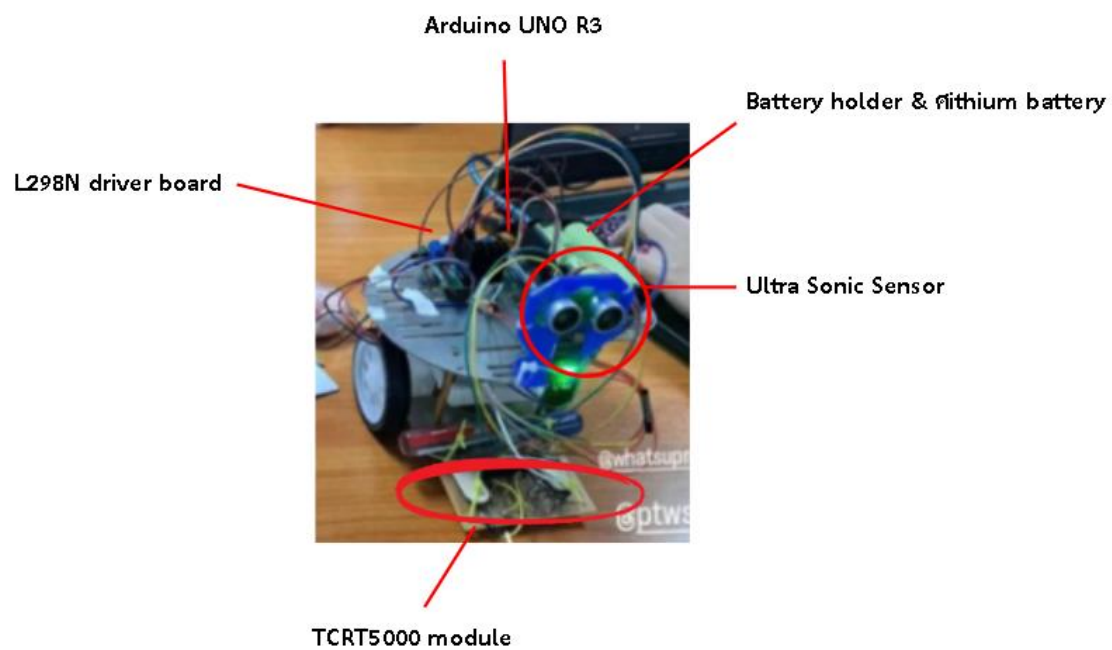


ภาพที่ 1 สนามแข่ง

Hardware Design

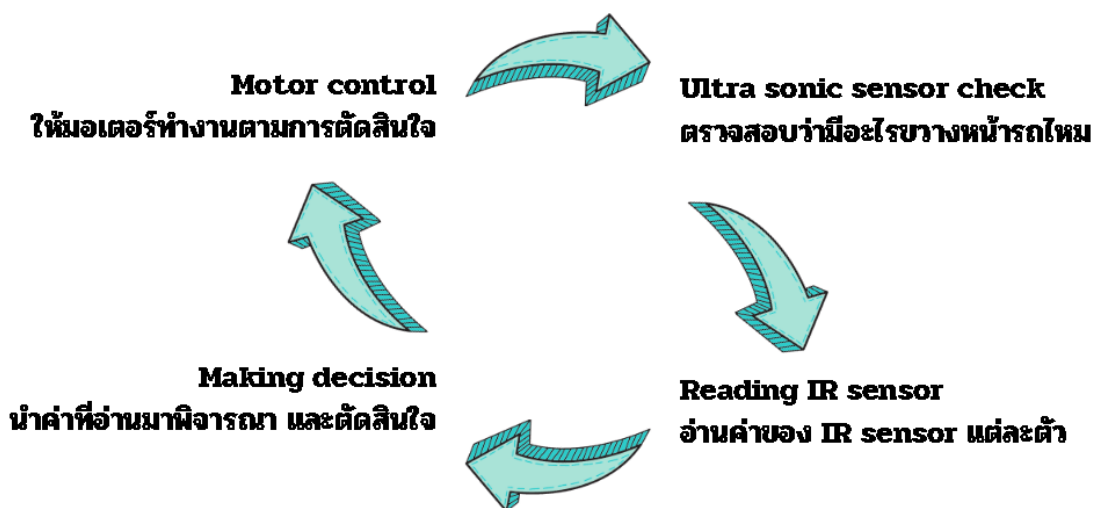
LDR Sensor จำเป็นต้องรับแสงอยู่บริเวณด้านหน้าของตัวรถเพื่ออ่านค่าแสงจากไฟแฟลช TCRT5000 ที่เป็น sensor ทำหน้าที่ตรวจจับเส้นเส้นต้องอยู่ใต้รถเพื่อให้รถเดินตามเส้นได้และ ต้องมีUltrasonic sensor อยู่หน้ารถเพื่อตรวจจับสิ่งกีดขวาง

Tools	Quantity
Arduino UNO R3	1
Car chassis with motors, wheels, and battery holder with lithium battery	1 set
L298N driver board	1
TCRT5000 module	5
220 Ohm resistors	5
10K Ohm resistors	5
Ultrasonic module	1
LDR sensor module	1
Breadboard	1
Jumper wires (Male to male & Male to female)	A lot



ภาพที่ 2 Robot car

หลักการทำงานของ Robot car Software design



ภาพที่ 4 หลักการทำงานของ Robot car

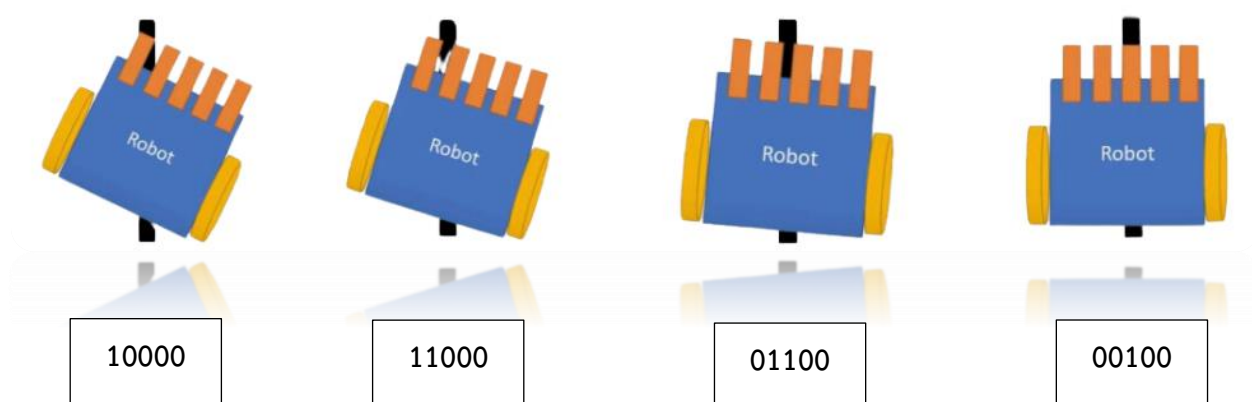
1. IR Sensor

หลักการทำงานของ TCRT5000 คือการใช้แสงอินฟราเรดเพื่อทำการตรวจจับพื้นผิว โดยระบบจะส่งแสงอินฟราเรดไปยังพื้นและวิเคราะห์ความเข้มของแสงที่สะท้อนกลับมา ตัวรับแสงจะมีการประเมินค่าที่แตกต่างกันตามความเข้มของแสงที่ได้รับ หากพื้นผิวมีสีขาว แสงที่สะท้อนกลับจะมีค่ามาก (ประมาณ 800-1000) แต่หากพื้นผิวเป็นสีดำ ค่าที่อ่านจะต่ำกว่า (ประมาณ 100-300)

ในขั้นตอนการวิเคราะห์ เซนเซอร์ทั้ง 5 ตัวจะอ่านค่าสัญญาณ Analog และแปลงค่าจากช่วง 0 - 1023 เป็นสัญญาณ Digital แทน (0 และ 1) ซึ่งจะแปลงเป็นสัญญาณดิจิทัล (digital) ให้อยู่ในรูปแบบบิต 0 และ 1 โดยมีการกำหนดเกณฑ์ (threshold) หากค่าที่อ่านได้มากกว่า หรือเท่ากับ 600 (แสดงว่าพื้นเป็นสีขาว) จะถูกแปลงเป็น 0 และหากค่าต่ำกว่า 600 (แสดงว่าพื้นเป็นสีดำ) จะถูกแปลงเป็น 1

ผลลัพธ์จากเซนเซอร์ทั้ง 5 ตัวจะถูกรวมเป็นข้อมูลในรูปแบบบิตไบนารีขนาด 5 บิต เช่น B00100 ซึ่งหมายความว่าเซนเซอร์กลางตรวจจับเส้นสีดำ แสดงว่ารถกำลังเคลื่อนที่ตรงไปข้างหน้า ในขณะที่ ถ้าค่ากลายเป็น B00110 หมายถึงการตรวจจับเส้นสีดำในเซนเซอร์ตัวกลาง และตัวขวา กลาง ก็ควรให้ทำการเลี้ยวขวานิดหน่อย เพื่อให้กลับไปวิ่งเป็นเส้นตรง

ข้อดีของวิธีการนี้คือสามารถตรวจจับพื้นผิวและตรวจการเปลี่ยนแปลงของเส้นได้อย่างแม่นยำ จะช่วยให้การควบคุมการเคลื่อนไหวยของรถทำได้มีประสิทธิภาพ นอกจากนี้ การแปลงค่าที่ได้เป็นบิตไบนารียังช่วยในการประมวลผลข้อมูลได้อย่างรวดเร็วมากขึ้น แต่ในขณะเดียวกัน ก็มีข้อเสียบางอย่างนั่นก็คือ การพึ่งพาการสะท้อนของแสงอาจทำให้เกิดความผิดพลาดในสภาพแวดล้อมที่มีแสงจ้าหรือพื้นผิวที่สะท้อนแสงแตกต่างกัน นอกจากนี้ยังต้องการการตั้งค่าที่เหมาะสมและการปรับจูนเพื่อให้ระบบทำงานได้อย่างแม่นยำในทุกสถานการณ์ ซึ่งควบคุมได้ค่อนข้างยาก



ภาพที่ 5 หลักการทำงานของ IR sensor

2. Robot car decision

2.1 PID Control (proportional integral derivative - Control) หรือการควบคุมแบบสัดส่วน-ปริพันธ์-อนุพันธ์ เป็นเทคนิคการควบคุมที่มีประสิทธิภาพในการรักษาความเสถียรภาพของระบบ โดยจะทำการคำนวณจากค่าความคลาดเคลื่อน (Error) ระหว่างค่าที่ต้องการ (Setpoint) และค่าที่ได้จริง (Process variable) ในระบบ

ในบริบทของ **Robot Car** ระบบ PID มีบทบาทสำคัญในการควบคุมการเคลื่อนที่ของรถให้สามารถเดินตามเส้นได้อย่างแม่นยำ โดยไม่ให้รถเอียงออกจากเส้นที่กำหนด เมื่อรถพบกับการเอียงหรือโค้งเล็กๆ เช่น อ่านได้ 01100 (รถเอียงขวานิดหน่อย) ระบบ PID ก็จะเลี้ยวซ้ายกลับไปนิดหน่อย เพื่อปรับทิศทางของรถให้กลับเข้ามาในเส้นทางที่ถูกต้อง

กระบวนการทำงานเริ่มต้นจากการกำหนดค่า **Base Speed** ซึ่งเป็นความเร็วพื้นฐานที่รถจะเคลื่อนที่ เมื่อเซนเซอร์อินฟราเรด (IR Sensor) อ่านค่าและตรวจจับความคลาดเคลื่อนจากเส้น จะทำการคำนวณค่า Error ตามเงื่อนไขที่ตั้งไว้สำหรับแต่ละระดับความเอียง

เมื่อคำนวณได้ค่า Error แล้ว ค่าดังกล่าวจะถูกนำเข้าสู่สูตรการควบคุม PID

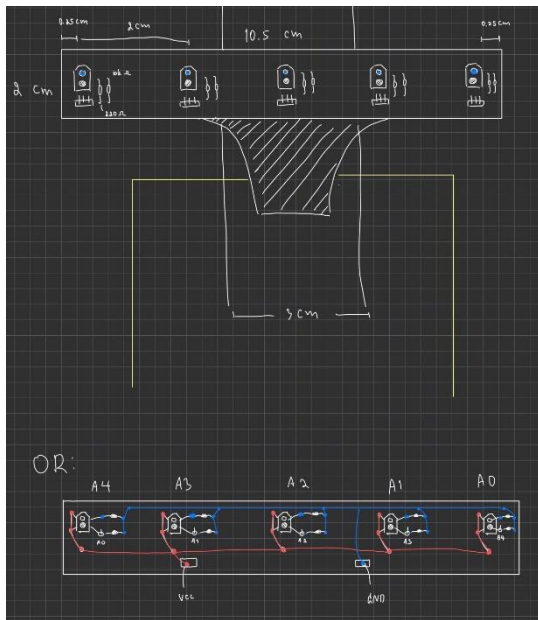
- K_p (ค่าคงที่สัดส่วน): กำหนดอัตราการตอบสนองตามค่าความคลาดเคลื่อนปัจจุบัน --> เป็นค่าที่ควรปรับให้สมดุลค่าแรก เพราะจะส่งผลให้เกิด overshoot ถ้าค่ามากเกินไป หรือถ้าน้อยเกินไป ก็จะไม่แข็งแรงพอ
- K_i (ค่าคงที่ปริพันธ์): รวมค่าความคลาดเคลื่อนในอดีตเพื่อแก้ไขการตอบสนองในระยะยาว --> จะนำส่วนของพื้นที่ใต้กราฟมาช่วยหักลบกัน เพื่อให้ได้พื้นที่เข้าใกล้ค่า 0 มากที่สุด หรือก็คือการวิ่งเป็นเส้นตรง
- K_d (ค่าคงที่อนุพันธ์): ใช้ในการคาดการณ์การเปลี่ยนแปลงของค่าความคลาดเคลื่อนในอนาคต --> สำคัญรองมาจากค่า K_p จะช่วยชะลอค่า K_p เมื่อใกล้กับจุดที่ค่าเป็น 0 แล้ว ทำให้เกิด overshoot น้อยลง



ภาพที่ 6 หลักการของ PID คร่าวๆ

*** ค่า K_p ถูกตั้งไว้ที่ 18.6, K_i อยู่ที่ 0.0 และ K_d ที่ 1.575 ***

ซึ่งค่า P, I และ D แต่ละตัวต้องใช้ค่า Error ในการคำนวณ และระยะห่างของ IR sensor ซึ่งได้ออกแบบไว้ ดังนี้



ภาพที่ 7 การจัดวาง IR sensor

	A4	A3	A2	A1	A0	Value
30° - 60°	0	0	1	0	1	-6
90°	0	0	1	1	1	-5
	0	0	0	0	1	-4
	0	0	0	1	1	-3
	0	0	0	1	0	-2
	0	0	1	1	0	-1
ตรง	0	0	1	0	0	0
	0	1	1	0	0	1
	0	1	0	0	0	2
	1	1	0	0	0	3
	1	0	0	0	0	4
90°	1	1	1	0	0	5
30° - 60°	1	0	1	0	0	6

ภาพที่ 8 ค่า Error ของแต่ละกรณี

```
PID_value = (Kp * P) + (Ki * I) + (Kd * D);
left_moter_speed = 80 - PID_value;
right_moter_speed = 80 + PID_value;
```

ภาพที่ 9 สูตรคำนวณ PID และควบคุมมอเตอร์

เมื่อคำนวณได้ค่าผลลัพธ์จากสูตร PID ระบบจะใช้ค่านี้ในการปรับความเร็วของล้อซ้ายและล้อขวา โดยอิงจาก Base Speed ที่ตั้งค่าไว้ที่ 80 ค่าที่คำนวณได้จะช่วยในการปรับความเร็วของล้อแต่ละด้านให้เหมาะสมกับทิศทางที่ต้องการ เพื่อให้รถสามารถกลับไปสู่เส้นทางที่ถูกต้องได้อย่างมีประสิทธิภาพ โดยรวมแล้ว PID Control ช่วยให้ Robot Car สามารถรักษาเส้นทางได้แม้ในสภาพแวดล้อมที่มีความซับซ้อนและเปลี่ยนแปลงตลอดเวลา ซึ่งเป็นหลักการที่สำคัญในการพัฒนาระบบควบคุมอัตโนมัติในงานวิศวกรรมและหุ่นยนต์ในปัจจุบัน

*** เมื่อรถออกนอกเส้นทางมันจะทำการหมุนวนรอบตัวเอง ตามเข็มนาฬิกา จาก PID ที่สั่งการมอเตอร์ ***

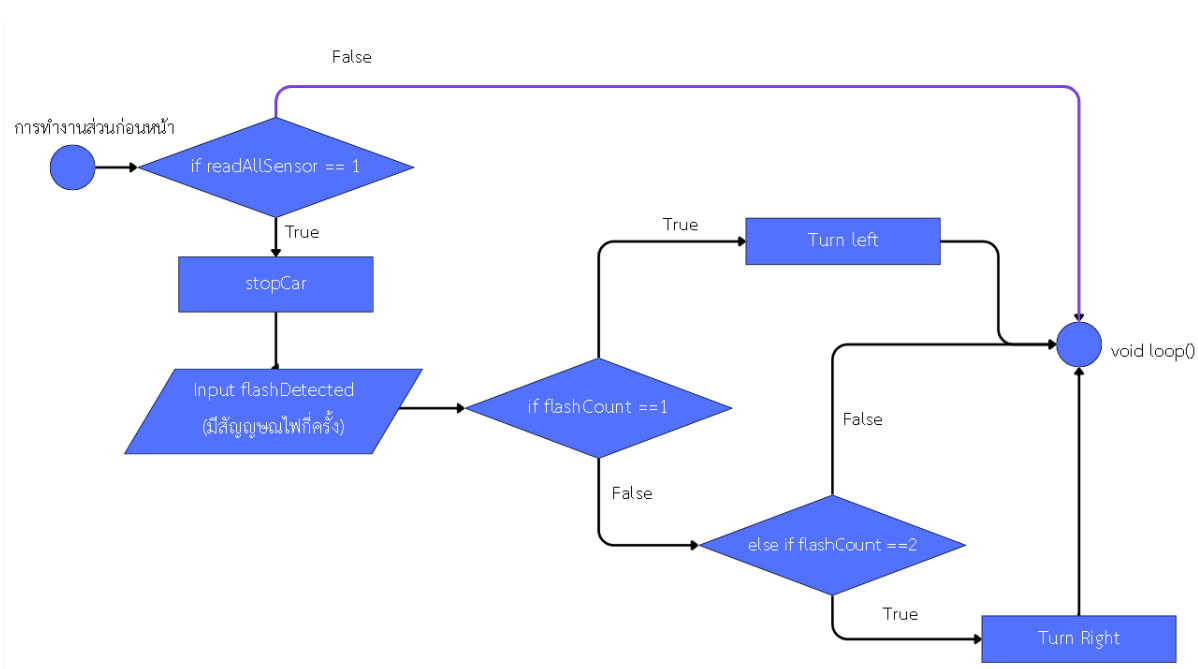
2.2 กรณีหยุดรถ

การหยุดรถ เกิดขึ้นจากสองกรณี ด้วยกันคือ

1. IR sensor อ่านได้ 11111 หรือก็คือเจอสีดำหมดซึ่งรถจะหยุดเป็นเวลา 5 วินาที หลังจากนั้นก็จะกลับมาอ่านค่าจากเซนเซอร์อีกรอบหนึ่ง ถ้าไม่เจอสัญญาณไฟกระพริบ
2. มีสิ่งกีดขวางมาขวางทาง จะใช้ Ultrasonic ในการควบคุม และสั่งการให้รถหยุด

1) LDR sensor

หลังจากรถหยุดแล้ว จะรอสัญญาณไฟ ถ้ามีสัญญาณไฟเข้ามา จำนวน 1 รอบ ก็จะทำให้การนับจำนวนว่ามีสัญญาณไฟแล้วหนึ่งรอบ จากฟังก์ชัน `int detectFlashes()` ที่เขียนไว้ซึ่งจะ return `flashCount` เป็นจำนวนครั้งของสัญญาณไฟ และก็จะรอ เป็นเวลา 1.75 วินาที เพื่อรอสัญญาณไฟรอบถัดไป ถ้าไม่มีสัญญาณไฟรอบถัดไป ก็จะนับได้ 1 ครั้ง ซึ่งสัญญาณไฟ เท่ากับหนึ่งรอบ ก็จะเลี้ยวซ้าย ถ้าเท่ากับสองรอบ หรือมากกว่าสองรอบ จะเลี้ยวขวา



ภาพที่ 10 แผนภาพ flowchart การทำงานของ LDR

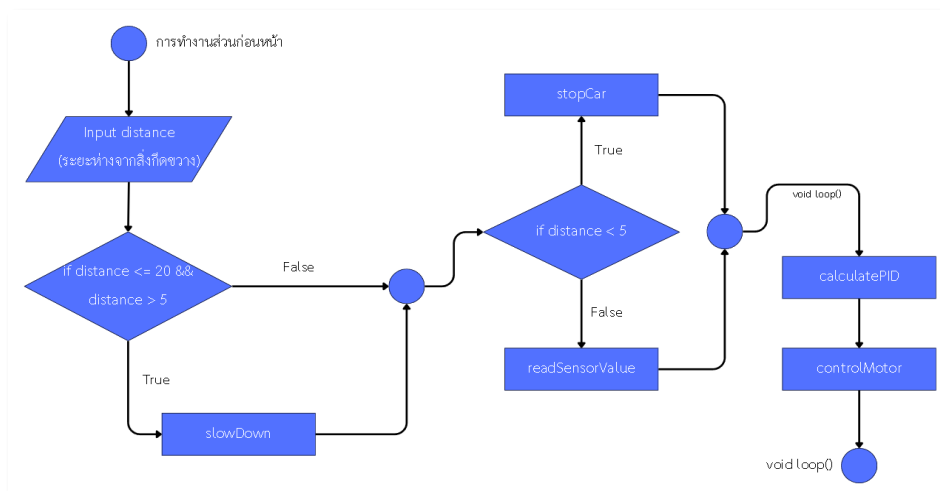
2) Ultrasonic sensor

การอ่านระยะทางโดยใช้ Ultrasonic sensor ใช้หลักการสะท้อนเสียง ซึ่งทำงานตามขั้นตอนดังนี้

1. การยิงเสียง: เซนเซอร์จะส่งสัญญาณเสียงที่มีความถี่สูงเป็นเวลาสั้น ๆ ประมาณ 10 มิลลิวินาที (ms) โดยเริ่มจากการตั้งค่า Trig Pin เป็น LOW เพื่อเคลียร์ค่า จากนั้นจึงเปลี่ยนเป็น HIGH เพื่อสร้างเสียงพัลส์ออกไป
2. การรอการสะท้อน: หลังจากเสียงถูกยิงออกไป เซนเซอร์จะรอเพื่อรับการสะท้อนเสียงกลับจากวัตถุที่อยู่ในเส้นทาง สัญญาณสะท้อนนี้จะถูกอ่านผ่าน Echo Pin โดยการใช้ฟังก์ชัน pulseIn() เพื่อวัดระยะเวลาที่เสียงใช้ในการกลับมาที่เซนเซอร์ ซึ่งเวลานี้จะถูกเก็บไว้ในตัวแปร duration ในหน่วยไมโครวินาที (μs)
3. การคำนวณระยะทาง: เมื่อตัวแปร duration ได้รับค่าแล้ว จะมีการคำนวณระยะทาง (distance) จากเวลาที่ได้รับ โดยใช้สูตร:

$$\text{distance} = \text{duration} * 0.034 / 2$$

โดยที่ 0.034 คือค่าความเร็วของเสียงในอากาศ (เซนติเมตรต่อไมโครวินาที) และการหารด้วย 2 เป็นการคำนึงถึงการเดินทางไปและกลับของเสียง การใช้เทคนิคนี้ช่วยให้สามารถวัดระยะทางได้อย่างแม่นยำจากเซนเซอร์ Ultrasonic ทำให้มีการประยุกต์ใช้งานในหลาย ๆ ด้าน เช่น ในหุ่นยนต์สำหรับหลีกเลี่ยงอุปสรรค หรือในการวัดระยะทางทั่วไปในระบบอัตโนมัติ การอ่านค่าจากเซนเซอร์ Ultrasonic จึงเป็นวิธีที่มีประสิทธิภาพในการตรวจสอบระยะห่างจากวัตถุ โดยสามารถให้ข้อมูลในเวลาที่ยรวดเร็วและมีความแม่นยำสูงในสภาพแวดล้อมที่หลากหลายค่ะ



ภาพที่ 11 แผนภาพ flowchart การทำงานของ Ultrasonic

3. การสั่งการมอเตอร์

บอร์ดควบคุมมอเตอร์ L298N เป็นอุปกรณ์ที่ใช้ในการควบคุมการหมุนของมอเตอร์ไฟฟ้า โดยสามารถควบคุมทั้งทิศทางการหมุนและความเร็วได้อย่างแม่นยำ มีกระบวนการทำงาน ดังนี้

การควบคุมทิศทางของมอเตอร์จะใช้พินที่เรียกว่า leftFront และ leftBack สำหรับล้อด้านซ้าย:

- หากจ่ายไฟ HIGH ที่พิน leftFront มอเตอร์จะหมุนไปข้างหน้า
- หากจ่ายไฟ HIGH ที่พิน leftBack มอเตอร์จะหมุนไปข้างหลัง

สำหรับล้อด้านขวา การควบคุมทิศทางจะทำได้โดยการใช้พิน rightFront และ rightBack ซึ่งทำงานในลักษณะเดียวกันตามที่อธิบายข้างต้น

***** ทิศทางการหมุนขึ้นกับการต่อขั้วของมอเตอร์ หากขั้วไม่ตรงจะทำให้หมุนทิศตรงข้ามกับที่กล่าวไว้ *****

การควบคุมความเร็วของมอเตอร์แต่ละตัวจะผ่านพิน EnA สำหรับล้อซ้าย และ EnB สำหรับล้อขวา โดยใช้สัญญาณ PWM (Pulse Width Modulation) ที่มีค่าอยู่ในช่วง 0-255:

- ค่า PWM ที่สูงขึ้นจะทำให้มอเตอร์หมุนด้วยความเร็วที่สูงขึ้น ขณะที่ค่าต่ำจะลดความเร็วของมอเตอร์ลง

การหมุนของมอเตอร์ขึ้นอยู่กับ การต่อขั้วที่ถูกต้อง หากขั้วต่อไม่ถูกต้อง มอเตอร์อาจจะหมุนในทิศทางตรงกันข้ามจากที่กำหนดไว้ ทำให้ต้องมีการตรวจสอบการต่อขั้วอย่างละเอียดก่อนการใช้งาน

โดยรวมแล้ว บอร์ด L298N เป็นเครื่องมือที่มีความยืดหยุ่นสูงในการควบคุมมอเตอร์ไฟฟ้าในหุ่นยนต์หรือระบบอัตโนมัติอื่น ๆ การควบคุมที่ชัดเจนทั้งในด้านทิศทางและความเร็วทำให้สามารถใช้งานได้มีประสิทธิภาพในการสร้างระบบขับเคลื่อนที่ซับซ้อนยิ่งขึ้น

```
if (stop) {
  digitalWrite(leftFront, LOW);
  digitalWrite(leftBack, LOW);
  digitalWrite(rightFront, LOW);
  digitalWrite(rightBack, LOW);
  stop = false;
}
```

ภาพที่ 12 ตัวอย่างการสั่งให้หยุดรถ

```
} else {
  digitalWrite(leftFront, HIGH);
  digitalWrite(leftBack, LOW);
  digitalWrite(rightFront, HIGH);
  digitalWrite(rightBack, LOW);
}
```

ภาพที่ 13 ตัวอย่างการสั่งให้รถไปข้างหน้า

ปัญหาที่พบบ่อย

1. เมื่อตอนซ่อมวีก่อนสอบไฟตอนนั้นค่อนข้างน้อยเพราะแบตเตอรี่หมดทำให้มอเตอร์วิ่งช้ากว่าปกติ ทำให้เวลาสอบจริงถ้าเอาถ่านไปชาร์จไฟจนเต็มแล้วเอามาวิ่งใหม่อาจมีปัญหาวิ่งได้ไม่เหมือนเดิมเพราะไฟมากขึ้น มอเตอร์หมุนเร็วกว่าตอนซ่อม โดยเฉพาะถ้าใช้ PID จะต้องไปปรับจูนค่าใหม่อีกทีหนึ่ง

แก้ปัญหาโดยการ ควรใช้มัลติมิเตอร์ในการวัด การจ่ายไฟของแบตเตอรี่ แล้วค่อยชาร์จแบตเตอรี่ให้ใกล้เคียงมากที่สุด หรือ ชาร์จถ่านเพิ่มนิดหน่อย เพื่อให้ใกล้เคียงกับค่าเดิม และระวังอย่าซ่อมจนถ่านหมดก่อนวันสอบ

2. TCRT5000 มีการอ่านค่าที่ผิดพลาดเพราะตำแหน่งของ TCRT5000 อยู่สูงจากระดับพื้นเกินไป

แก้ปัญหาโดยการ ขยับ TCRT5000ลงมาด้านล่างอีกและตรวจสอบค่าดูจนกว่าจะเจอตำแหน่งที่เหมาะสม แต่ค่อนข้างควบคุมได้ยาก เนื่องจากน้ำหนักของตัวรถที่ส่งผล และการติดตั้งตัวเซนเซอร์ ทำให้ตำแหน่งมันลากพื้น อาจจะต้องหาตัวอะไรมาค้ำให้ไม่ต่ำเกินไปจนติดกับพื้น

3. ติดตั้ง Sensor ต่ำเกินไปทำให้เวลาวิ่งในสนามจะมีจุดที่ใช้เทปกาวแปะเอาไว้มีปัญหา เช่น อาจจะติดหรือชนเมื่อเวลาเข้าโค้งทำให้หลุดโค้งออกไป

แก้ปัญหาโดยการ ปรับระดับขึ้นมาอีกเพื่อไม่ให้ติดหรือชน หาอะไรมาค้ำ หรือถ่วงอีกฝั่งของรถ

4. การปรับจูน PID ที่ความเร็วรถสูงๆทำได้ยากเพราะต้องใช้เวลาปรับจูนนานประกอบกับมีผู้ใช้งานสนามค่อนข้างเยอะทำให้ต้องต่อคิวรอ

แก้ปัญหาโดยการ ปรับลดความเร็วรถลงมาเพื่อให้เหมาะสมกับเวลาที่เหลืออยู่

5. บางทีเมื่อรถเจอเส้นดำหมดแล้วรถไม่หยุดเพราะความเร็วรถเร็วเกินไปทำให้หยุดไม่ทัน แล้วเลยไปเจอเส้นทางอื่น หรือ ในบางครั้งลดความเร็วมอเตอร์ลงแล้วแต่ก็ยังไม่หยุดเพราะในระหว่างที่หยุดรถยังไถลไปได้นิดหนึ่ง แล้ว TCRT5000 ไปตรวจเจอกรณีอื่นที่ไม่ใช่ดำหมดทำให้รถเลี้ยว จะต้องเจอตำแหน่งแบบพอดีรถไม่ไถลเลยอย่างเดียวถึงจะหยุด

แก้ปัญหาโดยการ เมื่อเจอตำแหน่งให้ติดอยู่ใน loop เป็นเวลา 5 วินาทีเพื่อรอสัญญาณไฟโดยไม่ต้องไปอ่านค่า Sensor เพราะถ้าอ่านอาจทำให้รถเลี้ยวไปเลยแบบไม่หยุดในกรณีที่จอดไม่ตรงกับเส้นดำแบบพอดี

Source code

```

4 // Arduino Line Follower Robot Code
5 const int photoResistorPin = A5; // ขาเชื่อมต่อของ photoresistor
6 const int trigPin = 12; // ขา Trig
7 const int echoPin = 13; // ขา Echo
8 float Kp = 18.6, Ki = 0.0, Kd = 1.575; //0.012 5
9 float error = 0, P = 0, I = 0, D = 0, PID_value = 0;
10 float I_max = -10; // ค่าขีดจำกัดสูงสุดสำหรับ I
11 float I_min = 10; // ค่าขีดจำกัดต่ำสุดสำหรับ I
12 float previous_error = 0;
13 float previous_D = 0; // เก็บค่า D ก่อนหน้าเพื่อใช้ใน low-pass filter
14 float alpha = 0.1; // ค่าคงที่สำหรับ low-pass filter (ระหว่าง 0 ถึง 1)
15 int sensorValues[5]; // ตัวแปรเก็บค่าที่อ่านจากเซ็นเซอร์
16 int digitalValues[5]; // ตัวแปรเก็บค่าที่แสดงเป็น digital (0 หรือ 1)
17 int initial_moter_speed = 80;
18
19 unsigned long previousMillis = 0; // ตัวแปรสำหรับเก็บเวลา
20 unsigned long previousTime = 0; // ตัวแปรสำหรับเก็บเวลา
21 bool stop = false;
22 bool slow = false;
23 // const long interval = 2000; // เวลาหน่วง (2000 มิลลิวินาที)
24
25 void read_sensor_value(void);
26 void calculate_pid(void);
27 void motor_control(void);
28
29 #define enLeft 6 //enLeftble1 L293 Pin enLeft
30 #define leftFront 8 //Motor1 L293 Pin leftFront
31 #define leftBack 9 //Motor1 L293 Pin leftBack
32
33 #define enRight 5 //enLeftble2 L293 Pin enRight
34 #define rightFront 10 //Motor2 L293 Pin leftFront
35 #define rightBack 11 //Motor2 L293 Pin leftBack
36
37
38 void setup() {
39   Serial.begin(9600);
40
41   pinMode(enLeft, OUTPUT);
42   pinMode(leftFront, OUTPUT);
43   pinMode(leftBack, OUTPUT);
44
45   pinMode(enRight, OUTPUT);
46   pinMode(rightFront, OUTPUT);
47   pinMode(rightBack, OUTPUT);
48
49   pinMode(trigPin, OUTPUT); // ตั้งค่า Trig เป็น OUTPUT
50   pinMode(echoPin, INPUT); // ตั้งค่า Echo เป็น INPUT
51 }
52
53 int detectFlashes() {
54   int flashCount = 0;
55   unsigned long startTime = millis(); // เก็บเวลาที่เริ่มต้นตรวจจับแฟลช
56   int threshold = 400; // ค่าความเข้มแสงที่มองว่ามีแฟลช (ปรับตามความเหมาะสม)
57   int ldrValue, lastldrValue;
58   bool flashActive = false;
59   unsigned long lastFlashTime = 0; // เวลาเมื่อแฟลชถูกตรวจจับครั้งสุดท้าย
60   int debounceDelay = 300; // หน่วงเวลาหลังตรวจจับแฟลช (300ms ป้องกันการตรวจจับซ้ำ)
61
62   lastldrValue = analogRead(photoResistorPin); // อ่านค่าเริ่มต้นจาก LDR
63
64   while (millis() - startTime < 5000) { // รอการตรวจจับแฟลชเป็นเวลา 5 วินาที
65     ldrValue = analogRead(photoResistorPin); // อ่านค่าปัจจุบันจาก LDR
66
67     // ตรวจจับการเปิดแฟลชจากการที่ค่าความเข้มแสงลดลงต่ำกว่า threshold (มีแฟลช)
68     if (ldrValue < threshold && !flashActive && millis() - lastFlashTime > debounceDelay) {
69       flashActive = true; // ตั้งค่าว่ามีแฟลชเปิดอยู่
70       flashCount++;
71       Serial.println("Flash detected!");
72       lastFlashTime = millis(); // เก็บเวลาที่ตรวจจับแฟลช
73     }
74     // ตรวจจับการปิดแฟลช (ค่าความเข้มแสงกลับมากกว่า threshold)
75     else if (ldrValue >= threshold && flashActive) {
76       flashActive = false; // ตั้งค่าว่าแฟลชปิดแล้ว
77     }
78
79     lastldrValue = ldrValue; // เก็บค่าล่าสุดไว้เพื่อเปรียบเทียบในรอบถัดไป
80   }
81
82   return flashCount;
83 }

```

setขา pin และค่าตัวแปรต่างๆ

set pin mode

สร้างฟังก์ชันสำหรับตรวจจับแฟลช

ภาพที่ 14 Source code


```

๐๖
84 void loop() {
85     long duration, distance;
86
87     // เคลียร์ค่า Trig Pin
88     digitalWrite(trigPin, LOW);
89     delayMicroseconds(2);
90
91     // ส่ง Pulse
92     digitalWrite(trigPin, HIGH);
93     delayMicroseconds(10);
94     digitalWrite(trigPin, LOW);
95
96     // อ่านค่า Echo Pin
97     duration = pulseIn(echoPin, HIGH);
98
99     // คำนวณระยะทาง (cm)
100    distance = duration * 0.034 / 2;
101
102    if (distance <= 20 && distance > 5) {
103        Serial.println("SLOW");
104        slow = true;
105    }
106    if (distance <= 5) {
107        Serial.println("STOP");
108        stop = true;
109    } else {
110        read_sensor_value();
111    }
112    calculate_pid();
113    motor_control();
114 }
115

```

สำหรับอ่านค่าจาก Ultrasonic และสั่งชะลอและหยุดรถ

สำหรับการเดินตามเส้นโดยจะอ่านค่าจาก Sensor แล้วคำนวณ PID
หลังจากนั้นสั่งมอเตอร์ทำงานตาม PID

```

116 void read_sensor_value() {
117     sensorValues[0] = analogRead(A0); // อ่านค่าจากเซ็นเซอร์ IR
118     sensorValues[1] = analogRead(A1); // อ่านค่าจากเซ็นเซอร์ IR
119     sensorValues[2] = analogRead(A2); // อ่านค่าจากเซ็นเซอร์ IR
120     sensorValues[3] = analogRead(A3); // อ่านค่าจากเซ็นเซอร์ IR
121     sensorValues[4] = analogRead(A4); // อ่านค่าจากเซ็นเซอร์ IR
122
123     for (int i = 0; i < 5; i++) {
124         // ปรับค่า analog เป็น digital ตามเงื่อนไข
125         if (sensorValues[i] >= 600) {
126             digitalValues[i] = 0; // digital = 0 สำหรับพื้นที่สีขาว
127         } else if (sensorValues[i] <= 599) {
128             digitalValues[i] = 1; // digital = 1 สำหรับพื้นที่สีดำ
129         }
130     }
131
132     for (int i = 0; i < 5; i++) {
133         Serial.print(sensorValues[i]);
134         Serial.print("\t");
135     }
136     Serial.println();

```

อ่านค่าจาก Sensor และปรับค่าตามพื้นที่สีขาว/ดำ

ภาพที่ 15 Source code

```

138 if ((digitalValues[4] == 1) && (digitalValues[3] == 1) && (digitalValues[2] == 1) && (digitalValues[1] == 1) && (digitalValues[0] == 1)) {
139   Serial.println("CarStop");
140   stop = true;
141   analogWrite(enLeft, 90);
142   analogWrite(enRight, 90);
143   digitalWrite(leftFront, LOW);
144   digitalWrite(leftBack, LOW);
145   digitalWrite(rightFront, LOW);
146   digitalWrite(rightBack, LOW);
147
148   Serial.println("Intersection detected. Waiting for flash signal...");
149
150   // รอการตรวจจับแฟลช
151   int flashes = detectFlashes();
152
153   if (flashes == 1) {
154     Serial.println("Turning left based on 1 flash.");
155     digitalWrite(leftFront, LOW);
156     digitalWrite(leftBack, LOW);
157     digitalWrite(rightFront, HIGH);
158     digitalWrite(rightBack, LOW);
159     delay(1750); // รอไฟเขียวเสร็จ
160   } else if (flashes >= 2) {
161     Serial.println("Turning right based on 2 flashes.");
162     digitalWrite(leftFront, HIGH);
163     digitalWrite(leftBack, LOW);
164     digitalWrite(rightFront, LOW);
165     digitalWrite(rightBack, LOW);
166     delay(1750); // รอไฟเขียวเสร็จ
167   }

```

ถ้าเจอตำแหน่งให้หยุดรถ

รอการตรวจจับแฟลชแล้วถ้าเจอแฟลช 1 ครั้ง จะเลี้ยวซ้าย
2 ครั้งจะเลี้ยวขวา

```

} else if ((digitalValues[4] == 0) && (digitalValues[3] == 0) && (digitalValues[2] == 0) && (digitalValues[1] == 0) && (digitalValues[0] == 1)) {
  Serial.println("Case-4");
  error = -4;
} else if ((digitalValues[4] == 0) && (digitalValues[3] == 0) && (digitalValues[2] == 0) && (digitalValues[1] == 1) && (digitalValues[0] == 1)) {
  Serial.println("Case-3");
  error = -3;
} else if ((digitalValues[4] == 0) && (digitalValues[3] == 0) && (digitalValues[2] == 0) && (digitalValues[1] == 1) && (digitalValues[0] == 0)) {
  Serial.println("Case-2");
  error = -2;
} else if ((digitalValues[4] == 0) && (digitalValues[3] == 0) && (digitalValues[2] == 1) && (digitalValues[1] == 1) && (digitalValues[0] == 0)) {
  Serial.println("TurnRight");
  error = -1;
} else if ((digitalValues[4] == 0) && (digitalValues[3] == 0) && (digitalValues[2] == 1) && (digitalValues[1] == 0) && (digitalValues[0] == 0)) {
  Serial.println("Go");
  error = 0;
} else if ((digitalValues[4] == 0) && (digitalValues[3] == 1) && (digitalValues[2] == 1) && (digitalValues[1] == 0) && (digitalValues[0] == 0)) {
  Serial.println("TurnLeft");
  error = 1;
} else if ((digitalValues[4] == 0) && (digitalValues[3] == 1) && (digitalValues[2] == 0) && (digitalValues[1] == 0) && (digitalValues[0] == 0)) {
  Serial.println("Case2");
  error = 2;
} else if ((digitalValues[4] == 1) && (digitalValues[3] == 1) && (digitalValues[2] == 0) && (digitalValues[1] == 0) && (digitalValues[0] == 0)) {
  Serial.println("Case3");
  error = 3;
} else if ((digitalValues[4] == 1) && (digitalValues[3] == 0) && (digitalValues[2] == 0) && (digitalValues[1] == 0) && (digitalValues[0] == 0)) {
  Serial.println("Case4");
  error = 4;
} else if ((digitalValues[4] == 1) && (digitalValues[3] == 1) && (digitalValues[2] == 1) && (digitalValues[1] == 0) && (digitalValues[0] == 0)) {
  Serial.println("Case5"); // 90
  error = 5;
} else if ((digitalValues[4] == 0) && (digitalValues[3] == 0) && (digitalValues[2] == 1) && (digitalValues[1] == 1) && (digitalValues[0] == 1)) {
  Serial.println("Case-5"); // 90
  error = -5;
} else if (digitalValues[0] == 1) {
  Serial.println("Case-6");
  error = -6;
} else if (digitalValues[4] == 1) {
  Serial.println("Case6");
  error = 6;
}

```

กำหนดค่าerror

ภาพที่ 16 Source code

```

209
210 void calculate_pid() {
211     P = error;
212     unsigned long currentTime = millis();
213     unsigned long deltaTime = currentTime - previousTime;
214     if (deltaTime == 0) deltaTime = 1; // ป้องกันการหารด้วยศูนย์
215     I += (error * deltaTime);
216     // จำกัดค่า I ไม่ให้สูงหรือต่ำเกินไป
217     I = constrain(I, I_min, I_max);
218     if (error == 0) {
219         I = 0;
220     }
221     // kp น้อย หลุดโค้ง kp มาก เสียเยอะ สำมาก kd ใช้ด้านถ้า kp เสียมากไป ดูค่า error ki ทำให้ตรงใกล้ 0 คำนวณ PID จาก error
222     // คำนวณค่า Derivative term
223     D = (error - previous_error) / deltaTime;
224
225     // ใช้ Low-pass filter เพื่อลด noise ในค่า D
226     D = alpha * D + (1 - alpha) * previous_D;
227
228     PID_value = (Kp * P) + (Ki * I) + (Kd * D);
229
230     previousTime = currentTime;
231     previous_error = error;
232     previous_D = D;
233 }
234

```

```

235 void motor_control() {
236     float left_moter_speed, right_moter_speed;
237     if (slow) {
238         left_moter_speed = 80 - PID_value;
239         right_moter_speed = 80 + PID_value;
240         slow = false;
241     } else {
242         left_moter_speed = initial_moter_speed - PID_value;
243         right_moter_speed = initial_moter_speed + PID_value;
244     }
245
246     left_moter_speed = constrain(left_moter_speed, 0, 255);
247     right_moter_speed = constrain(right_moter_speed, 0, 255);
248
249     analogWrite(enLeft, left_moter_speed);
250     analogWrite(enRight, right_moter_speed);
251
252     if (stop) {
253         digitalWrite(leftFront, LOW);
254         digitalWrite(leftBack, LOW);
255         digitalWrite(rightFront, LOW);
256         digitalWrite(rightBack, LOW);
257         stop = false;
258     } else {
259         digitalWrite(leftFront, HIGH);
260         digitalWrite(leftBack, LOW);
261         digitalWrite(rightFront, HIGH);
262         digitalWrite(rightBack, LOW);
263     }
264 }
265

```

นำค่าจากการคำนวณ PID มาปรับความเร็ว เพื่อควบคุมมอเตอร์

สั่งมอเตอร์หยุดทำงานเมื่อได้สัญญาณว่าให้หยุด

สั่งมอเตอร์เดินไปข้างหน้าต่อไปเมื่อไม่มีสัญญาณบอกให้หยุด

ภาพที่ 17 Source code